Практическая работа №2

Аудит готового клиент-серверного приложения

ФИО: Бражников Алексей Евгеньевич

Цель работы: изучить клиент-серверное приложение на Python, проверить обмен данными по TCP и UDP, исследовать ошибки и сравнить JSON с XML.

1 Ответы на вопросы

1. Какой протокол используется в send_reliable?

Используется TCP: для сокета IPv4 указан тип SOCK_STREAM. Вызов connect() устанавливает соединение с сервером перед передачей данных. TCP обеспечивает упорядоченную передачу байтов и повторную отправку потерянных данных, пока соединение работоспособно.

2. Какой протокол используется в send_fast?

Используется UDP: сокет IPv4 создаётся с типом SOCK_DGRAM. Функция sendto() отправляет отдельную датаграмму по указанному адресу без установки соединения. UDP не гарантирует доставку датаграмм и сохранение порядка их поступления.

3. В каком формате передаются данные?

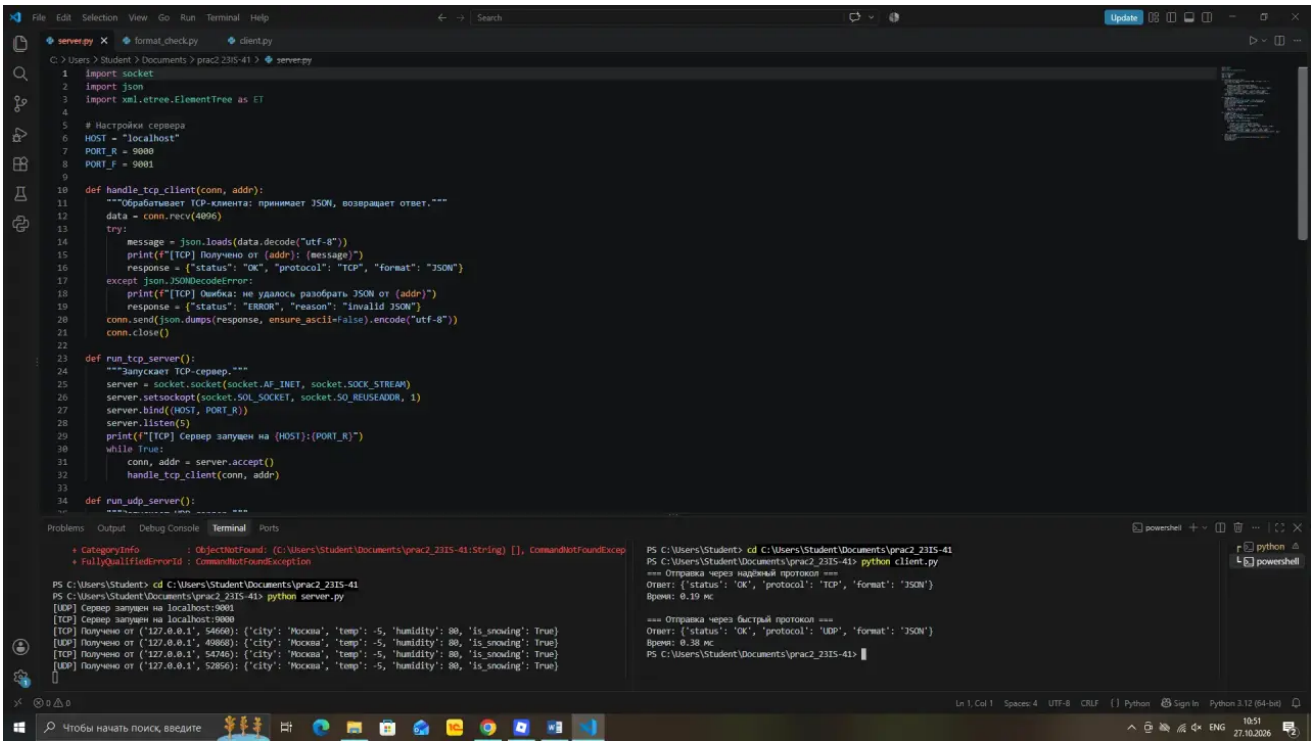
Данные передаются в формате JSON и кодируются в UTF-8. Функция json.dumps() преобразует объект weather в JSON-строку, а encode() превращает её в байты для отправки. Формат подтверждается следующей строкой из client.py:

```
client.send(json.dumps(message,
ensure_ascii=False).encode("utf-8"))
```

4. Что будет при отправке обычной строки hello?

Если отправить сырые байты b"hello", json.loads() вызовет JSONDecodeError. Сервер выведет сообщение об ошибке и вернёт {"status": "ERROR", "reason": "invalid JSON"}. После обработки ошибки TCP-соединение будет закрыто, а сервер продолжит ожидать новых клиентов.

2 Запуск сервера и клиента



В двух терминалах VS Code запущены server.py и client.py: сервер вывел сообщения о запуске и приёме данных о погоде по TCP и UDP. Клиент получил по обоим протоколам ответы со статусом OK, при этом на скриншоте время обмена составляет 0,19 мс для TCP и 0,38 мс для UDP.

3 Проверка граничных случаев

3.1 Сервер выключен и запущен TCP-клиент

После остановки сервера выполнен client.py. При вызове connect() программа завершилась с необработанным исключением:

```
ConnectionRefusedError:
[Errno 111] Connection refused
```

На порту 9000 отсутствует принимающий сервер, поэтому соединение отклоняется. UDP-часть при обычном запуске не выполняется, поскольку исключение TCP прерывает программу.

3.2 Сервер выключен и запущен UDP-клиент

Отдельно вызвана send_fast(weather) без TCP-части. Получен ответ {"status": "TIMEOUT"}; фактическое время ожидания составило **2002,25 мс**, то есть примерно 2 секунды.

UDP отправляет датаграмму без установки соединения, но ответ не приходит. Таймаут 2,0 секунды вызывает socket.timeout, который обработан в функции send_fast().

3.3 Клиент отправляет не JSON

При работающем сервере по TCP отправлены байты b"hello". Это эквивалентно временной замене строки отправки из задания; исходный файл клиента оставлен без изменений.

```
{"status": "ERROR", "reason": "invalid JSON"}
```

Сервер записал: «[TCP] Ошибка: не удалось разобрать JSON от …». Исключение JSONDecodeError обработано, клиент получил ответ об ошибке, сервер продолжил работу.

3.4 Сравнение форматов

Выполнен исходный format_check.py. Размер измерен для UTF-8 с теми пробелами и тегами, которые формирует программа.

Показатель	**JSON**	**XML**
Размер	72 байта	111 байт
Тип temp после чтения	int	str
Тип humidity после чтения	int	str
Тип is_snowing после чтения	bool	str

JSON компактнее на 39 байт: XML содержит повторяющиеся открывающие и закрывающие теги. В этой реализации JSON сохраняет числа и логический тип, а значения XML извлекаются как строки.

4 Выявленные недостатки приложения

Ошибочная полнота проверки форматов. Вывод «Данные совпадают: True» относится только к выражению restored_json == weather. Сравнение restored_xml == weather не выполняется; оно даст False, поскольку после чтения XML числа и логическое значение представлены строками.

XML не проверяется повторным разбором. Словарь restored_xml строится из уже существующего root. Для полноценной проверки следует разобрать xml_data через ET.fromstring(), восстановить типы и сравнить результат с weather.

Нет границ сообщений TCP. Единственный recv(4096) не гарантирует получение всего JSON: TCP передаёт поток байтов, и сообщение может прийти частями. Нужны длина сообщения либо разделитель и цикл чтения; вместо send() следует использовать sendall().

Нет таймаутов TCP и обработки сетевых ошибок. Ожидание клиента или сервера может затянуться. TCP-сервер обслуживает соединения последовательно, поэтому клиент, который подключился и ничего не отправляет, блокирует обработку остальных TCP-клиентов.

Не обработаны все ошибки входных данных. Перехватывается только JSONDecodeError. Некорректный UTF-8 вызывает UnicodeDecodeError; сетевые ошибки также могут прервать обработку. Это замечание установлено при чтении кода, отдельный тест повреждённого UTF-8 в обязательные проверки не входил.

Ограничен объём данных. В UDP принимается не более 4096 байт; большая датаграмма может быть усечена. В TCP тот же размер задаёт размер одного чтения, а не максимальную длину корректно собранного сообщения.

Проверяется только синтаксис JSON. Сервер не проверяет наличие city, temp, humidity и is_snowing, а также допустимость их значений. В рабочем приложении нужна проверка структуры и типов данных.

5 Вывод

Для этого приложения я выбрал бы **TCP и JSON**. Данные о погоде должны передаваться с контролем доставки и порядка байтов; при этом приложение всё равно должно обрабатывать разрыв соединения, таймауты и границы сообщений.

JSON в данном примере занимает меньше места и восстанавливает числа и логические значения без дополнительного преобразования. UDP подошёл бы для частых обновлений, где потеря отдельного сообщения допустима. В учебном приложении основной обмен и обработка некорректного JSON работают, но перечисленные недостатки снижают его надёжность.